

Vectorizar un Nuevo Corpus — StudIA

Versión: 1.0 · **Fecha:** 18/09/2026

Para: quien tenga que indexar un corpus académico propio (otra carrera, otro año, otro conjunto de materias).

La calidad de las respuestas depende más del corpus que del código. Este manual cubre el ciclo completo: organizar el material, indexarlo, sumarle OCR, vectorizarlo y verificar que quedó bien.

1. Organizar el material

El ingestor toma la **estructura de carpetas** como fuente de la materia, así que conviene ordenarlo antes de indexar:

```
DATA/
├── 1C - Analisis Matematico I/
│   ├── Apunte de catedra.pdf
│   └── TP1 resuelto.pdf
├── 2C - Fisica II/
└── 5C - Maquinas Electricas/
```

Recomendaciones:

- **Un nivel por materia**, con el cuatrimestre al principio del nombre. El selector las ordena por ese número (el campo es texto `"1C".."10C"` y se castea a entero para que `10C` no quede antes que `1C`).
- **Sacar lo que no es material de estudio**: instaladores, backups, carpetas de proyectos con código, fotos personales. El ingestor filtra buena parte, pero lo que no entra no hay que indexarlo.
- **Cuidar la privacidad**: listados de alumnos, planillas con datos personales y documentación administrativa no deberían entrar al índice. En el corpus original se excluyeron explícitamente los archivos con listados de nombres.
- **Rutas cortas**: Windows admite hasta 260 caracteres. Conviene no acercarse: en el corpus de referencia, ya reducido, la ruta más larga quedó en 219.

Formatos que se leen: **PDF, DOCX, PPTX, XLSX, TXT, MD, IPYNB**. Los formatos viejos de Office (`.doc` , `.ppt` , `.xls` , `.rtf` , `.odt`) se registran pero no se extraen: conviene convertirlos antes.

2. Ingesta

```
python tools/studia/ingest.py \
    --corpus "D:\FACULTAD\DATA" \
    --db      "D:\StudIA\studia.db"
```

Opción	Para qué
<code>--materia <texto></code>	Indexar sólo las materias cuyo nombre contenga ese texto
<code>--limpiar</code>	Borrar el índice y rehacerlo desde cero
<code>--archivo <ruta></code>	Sumar un documento suelto (lo usa la aplicación para la bibliografía propia)
<code>--quitar <ruta></code>	Quitar un documento del índice (no borra el archivo)
<code>--descartados <archivo></code>	Lista de rutas a excluir explícitamente

Es re-ejecutable. Saltea lo ya procesado —por huella MD5 de los primeros 4 MB más el tamaño— y **reintenta lo que falló**: si faltaba una librería, se instala y se vuelve a correr sin rehacer la corrida entera.

Nunca modifica el corpus. Se abre en modo sólo lectura.

Qué hace por dentro

1. Recorre la carpeta y descarta lo que no es material de estudio (`site-packages` , `__pycache__` , `*.dist-info` , metadata de paquetes, archivos de bloqueo `~$`).
2. Extrae el texto según el formato.
3. Parte el texto en fragmentos de **1.200 caracteres con 200 de solape** —el solape evita que una definición partida al medio quede irrecuperable—.
4. Escribe el índice **SQLite + FTS5** con `remove_diacritics 2` , para que "mecanica" encuentre "mecánica".
5. Los documentos sin texto extraíble se marcan `necesita_ocr` en vez de descartarse, de modo que se les pueda sumar OCR después sin re-ingestar el resto.

Cuánto tarda

Para el corpus de referencia (2.505 documentos indexados, 23 GB de origen) la ingesta completa lleva **varias horas**, dominada por la extracción de texto de los PDF grandes. Conviene correrla de a materias (`--materia`) la primera vez, para detectar problemas temprano.

3. Verificar el índice

```
python tools/studia/estado.py --db "D:\StudIA\studia.db" --materias
python tools/studia/estado.py --db "D:\StudIA\studia.db" --buscar "modelo OSI" -k 5
```

Informa cuántos documentos y fragmentos hay, el desglose por materia y —con `--buscar` — qué devuelve una consulta concreta **con su score**. Ese score es el insumo para calibrar la abstención (§7).

Auditar lo que quedó afuera

```
python tools/studia/listar_excluidos.py --corpus "D:\FACULTAD\DATA" --salida "D:\StudIA\auditoria"
```

Genera los CSV de qué se excluyó y por qué. Importa las reglas de `ingest.py` en vez de duplicarlas, así la auditoría no se desincroniza del ingestor.

4. OCR de los documentos escaneados

Un PDF escaneado es una imagen: no tiene texto que extraer. Esos documentos quedaron marcados `necesita_ocr` en la ingesta.

```
python tools/studia/ocr.py --db "D:\StudIA\studia.db"
```

Opción	Default	Para qué
<code>--materia</code>	todas	Acotar a una materia
<code>--dpi</code>	220	Resolución de rasterizado

Opción	Default	Para qué
<code>--procesos</code>	automático	Paralelismo
<code>--min-paginas</code> / <code>--max-paginas</code>	0	Acotar por tamaño del documento
<code>--limite</code>	0	Procesar sólo N documentos (prueba)

Requiere **Tesseract con el paquete de español**. Es el paso más lento de todo el proceso: conviene empezar con `--limite 5` para verificar la calidad antes de lanzarlo sobre cientos de documentos.

En el corpus de referencia el OCR se completó sobre todos los escaneados: los 2.505 documentos del índice quedaron en estado `ok`, ninguno sin texto extraído. Lo que el ingestor descarta queda igualmente registrado y auditable (§3).

5. Vectorización (búsqueda semántica)

Sin este paso StudIA funciona igual, con búsqueda por palabras. Con él encuentra por significado.

1. Levantar el servidor de embeddings:

```
llama-server -m bge-m3-Q8_0.gguf --embeddings --pooling cls --port 8081
```

`--pooling cls` es el que corresponde a bge-m3. Con otro, los vectores salen mal **sin dar error**.

2. Vectorizar el índice:

```
python tools/studia/vectorizar.py --db "D:\StudIA\studia.db" --url http://127.0.0.1:8081
```

Opción	Default	Para qué
<code>--materia</code>	todas	Acotar a una materia
<code>--presupuesto</code>	4096	Tokens por lote
<code>--max-fragmentos</code>	8	Fragmentos por pedido
<code>--limpiar</code>	—	Borrar los vectores y rehacerlos

Es **incremental y re-ejecutable**: sólo procesa lo que falta. Los vectores se guardan **normalizados** en la tabla `vectores` de la misma base —así el coseno es un producto escalar— y `vectores_info` registra dimensión y modelo. Los vectores de otra dimensión se ignoran en vez de producir resultados sin sentido.

3. Configurar la URL en el panel de StudIA (o dejar que la versión instalada levante el servidor sola).

6. Reducir y distribuir el corpus

Buena parte del material no alimenta el índice. Para armar una copia con sólo lo que sí:

```
python tools/studia/copiar_corpus.py \
--db "D:\StudIA\studia.db" \
--origen "D:\FACULTAD\DATA" \
--destino "D:\DATA_StudIA" \
--simular
```

`--simular` informa qué haría sin copiar nada. En el corpus de referencia la reducción fue de **23 GB a 5,93 GB (2.505 archivos)**.

Cómo se entrega

El material se arma en **una sola carpeta con dos cosas adentro**:

```
MiCarpeta    studia.db    el índice
DATA_StudIA\ los documentos originales
```

Esa carpeta se copia tal cual a la otra máquina —disco interno, externo o pendrive— y desde la aplicación se elige el `studia.db` con **Abrir índice**. De esa misma ruta sale también dónde están los documentos, porque se buscan en la `DATA_StudIA` hermana del índice.

La única condición es no separarlas. No hay scripts de copiado ni rutas que registrar: por eso el destino de `--destino` conviene que sea la carpeta que va a viajar, con el `studia.db` al lado.

7. Recalibrar la abstención — paso obligatorio

El umbral por defecto (`-7.0`) se calibró para un índice de cientos de miles de fragmentos. **Los scores de BM25 dependen del tamaño del corpus: un umbral de otro índice no sirve.**

1. Escribir **30 preguntas en lenguaje natural**: la mitad sobre temas cubiertos por el material, la mitad ajenas al corpus.
2. Correr cada una con `estado.py --buscar` y anotar el score.
3. Elegir el umbral que deja pasar todas las cubiertas y rechaza todas las ajenas.
4. Cargarlo en `umbralAbstencion` y correr la suite de pruebas.

Señales de que el umbral quedó mal:

Síntoma	Umbral
Se abstiene ante preguntas que el material sí cubre	demasiado exigente
Responde con fragmentos que no vienen al caso	demasiado permisivo

8. Lista de verificación

- ☐ Material organizado por materia, con el cuatrimestre al principio del nombre
- ☐ Material sensible o administrativo fuera del corpus
- ☐ Formatos viejos de Office convertidos

- [] `ingest.py` corrido y terminado sin errores pendientes
- [] `estado.py --materias` muestra todas las materias esperadas
- [] `listar_excluidos.py` revisado: nada importante quedó afuera
- [] `ocr.py` corrido sobre los `necesita_ocr`
- [] `vectorizar.py` completo, `vectores_info` con la dimensión correcta
- [] Umbral de abstención recalibrado con 30 preguntas
- [] `studia.db` y `DATA_StudIA` juntos en la carpeta que se va a entregar
- [] Índice abierto desde la aplicación con **Abrir índice**
- [] Prueba final: una pregunta cubierta responde con citas; una ajena se abstiene